

Data Architecture for Multi-Branch Retail Inventory

Benchmarking an eight-entity relational model against industry standards, and the analytics it makes possible

Yanyan Cai · MSc Business Analytics and Financial Technology, University of Dundee

Companion note to *Relational Database Design (AC51049)* · September 2026

1. EXECUTIVE SUMMARY

This note takes the eight-entity schema built for the AC51049 project (Supplier, Branch, Employee, Customer, Product, Inventory, Order, OrderDetails) and asks two questions that a coursework brief does not: **does this model resemble what the retail industry actually standardises on, and what analytical work does it unlock that a spreadsheet estate cannot support?**

On the first question, the eight entities map cleanly onto the core subject areas of the ARTS Operational Data Model, the retail industry's reference database design maintained by the Object Management Group and currently at version 7.3.¹ The schema is a simplification, not a deviation. The subject areas it omits, chiefly transaction-level point of sale, price and promotion management, and merchandise hierarchy, are identified in Section 5 as the natural next build.

On the second question, Section 4 sets out eight analyses the schema supports directly, each with a worked query: cross-branch stock position, reorder-point breach detection, ABC classification, stock turn and days of cover, supplier lead-time and fill-rate scoring, inter-branch imbalance, count-versus-system variance attribution, and branch-level demand analysis. Each of these is impossible to run reliably across five separate spreadsheets, and each becomes a single query once identity and referential integrity are enforced in one place.

Section 6 places this in the labour market: SQL appears in more data analyst job postings than any other named technical skill, ahead of Excel, Python and every BI tool.³

2. BENCHMARKING AGAINST THE INDUSTRY REFERENCE MODEL

The retail sector has a published reference data model. The ARTS Operational Data Model, originally developed by the Association for Retail Technology Standards under the National Retail Federation and now maintained by the Object Management Group's Retail Domain Technology Committee, is described by OMG as “a retail-specific roadmap for database design”.¹ Version 7.3 is the current release and ships with logical entity-relationship diagrams, ERwin models and a SQL Server sample database.¹

The relevance is straightforward: a schema is not credible because it is elegant, it is credible because it corresponds to how the domain is actually modelled. The mapping below is the honest one, including where the AC51049 model is thinner than the standard.

ARTS SUBJECT AREA	AC51049 COVERAGE	STATUS
Party (supplier, employee, customer)	Supplier, Employee, Customer as separate entities	Covered, though ARTS generalises these into a single Party supertype with role subtypes
Item / merchandise	Product	Covered at item level; no merchandise hierarchy (department, class, subclass)
Location	Branch	Covered
Inventory position	Inventory as a Product-by-Branch junction	Covered
Purchase order and fulfilment	Order, OrderDetails in header-and-line form	Covered
Retail transaction (POS)	Not modelled	Gap. The largest omission; see Section 5
Price and promotion	Not modelled	Gap
Inventory movement	Not modelled as an event table	Gap. Position is held, movement is not history

THE MOST IMPORTANT GAP, STATED PLAINLY

The schema stores inventory *position* (how much is here now) but not inventory *movement* (what changed it, when, and who did it). Without a movement event table, a discrepancy can be detected but not attributed. Section 5.2 sets out what that table would look like and why it is the single highest-value addition.

3. WHAT REFERENTIAL INTEGRITY BUYS, CONCRETELY

“Enforcing referential integrity” is a phrase that appears on a great many CVs and is rarely unpacked. What it means in this schema is a specific set of constraints, each of which makes a specific category of bad data impossible to write rather than merely unlikely.

```
-- Inventory cannot exist for a product or branch that does not exist.
-- ON DELETE RESTRICT is deliberate: deleting a product that still holds
-- stock should fail loudly, not silently orphan the stock record.
```

```
CREATE TABLE Inventory (
  ProductID      INT          NOT NULL,
  BranchID       INT          NOT NULL,
  QuantityOnHand INT          NOT NULL DEFAULT 0,
  ReorderPoint   INT          NOT NULL DEFAULT 0,
  LastCounted    DATE,
  CONSTRAINT PK_Inventory PRIMARY KEY (ProductID, BranchID),
  CONSTRAINT FK_Inv_Product FOREIGN KEY (ProductID)
    REFERENCES Product(ProductID) ON DELETE RESTRICT,
  CONSTRAINT FK_Inv_Branch  FOREIGN KEY (BranchID)
    REFERENCES Branch(BranchID)   ON DELETE RESTRICT,
  CONSTRAINT CK_Inv_NonNegative CHECK (QuantityOnHand >= 0)
);
```

Four design decisions in that fragment are worth defending, because each corresponds to a failure that occurs in spreadsheet estates:

DECISION	WHAT IT PREVENTS
Composite primary key (ProductID, BranchID)	The same product cannot hold two competing stock rows at the same branch. In a spreadsheet estate this duplication is the single most common source of an unreconcilable group stock figure.
Foreign keys to Product and Branch	Stock cannot be recorded against a product code or a branch that does not exist. Typed-in codes that nearly match are rejected at write time rather than discovered at year end.
ON DELETE RESTRICT rather than CASCADE	Deleting a discontinued product that still shows stock fails instead of quietly destroying the stock record. In inventory, silent deletion is worse than a blocked transaction.
CHECK (QuantityOnHand >= 0)	Negative stock, which is an accounting artefact rather than a physical state, cannot enter the table at all.

The header-and-line separation between Order and OrderDetails does comparable work on the purchasing side. In a single flat sheet, a ten-line order repeats the supplier, date and branch ten times, and a corrected supplier reference on nine of those ten rows produces an order that belongs to two suppliers at once. Third normal form removes the possibility.

Implementation note: `ORDER` is a reserved word in standard SQL. The table must be quoted as `"Order"` or, preferably, named `PurchaseOrder` or `Orders`. This is a small point that reliably breaks a first build.

4. WHAT THE SCHEMA LETS YOU ANALYSE

The argument for a relational model is not aesthetic. It is that a normalised schema turns questions that require a week of manual reconciliation into a single query. Eight examples follow, ordered roughly by how quickly they pay for themselves.

4.1 Cross-branch stock position for one product

The question a spreadsheet estate cannot answer without opening five files and trusting that all five use the same product code.

```
SELECT  b.BranchName,
        i.QuantityOnHand,
        i.ReorderPoint,
        i.QuantityOnHand - i.ReorderPoint AS Headroom
FROM    Inventory i
JOIN    Branch  b ON b.BranchID = i.BranchID
JOIN    Product p ON p.ProductID = i.ProductID
WHERE   p.SKU = 'SKU-10482'
ORDER BY Headroom;
```

4.2 Reorder-point breaches across the estate

The daily operational list. Ranking by shortfall rather than alphabetically is what makes it actionable.

```

SELECT  b.BranchName, p.SKU, p.ProductName,
        i.QuantityOnHand, i.ReorderPoint,
        i.ReorderPoint - i.QuantityOnHand AS Shortfall
FROM    Inventory i
JOIN    Branch  b ON b.BranchID = i.BranchID
JOIN    Product p ON p.ProductID = i.ProductID
WHERE   i.QuantityOnHand < i.ReorderPoint
ORDER BY Shortfall DESC;

```

4.3 ABC classification by revenue contribution

Pareto classification is the standard method for deciding where counting effort and safety stock should go. The ECR field study found that record-accuracy uplift concentrates in fast-moving A-class items, which makes this query the one that directs the accuracy programme.²

```

WITH line_value AS (
    SELECT  od.ProductID,
           SUM(od.Quantity * od.UnitPrice) AS Revenue
    FROM    OrderDetails od
    JOIN    "Order" o ON o.OrderID = od.OrderID
    WHERE   o.OrderDate >= DATE '2025-09-01'
    GROUP BY od.ProductID
),
ranked AS (
    SELECT  ProductID, Revenue,
           SUM(Revenue) OVER (ORDER BY Revenue DESC)
           / SUM(Revenue) OVER () AS CumulativeShare
    FROM    line_value
)
SELECT  p.SKU, p.ProductName, r.Revenue,
        ROUND(r.CumulativeShare * 100, 1) AS CumPct,
        CASE WHEN r.CumulativeShare <= 0.80 THEN 'A'
              WHEN r.CumulativeShare <= 0.95 THEN 'B'
              ELSE 'C' END AS ABCClass
FROM    ranked r
JOIN    Product p ON p.ProductID = r.ProductID
ORDER BY r.Revenue DESC;

```

4.4 Stock turn and days of cover by branch

The working-capital view. Two branches with identical stock values can have very different cash positions once velocity is taken into account.

```

SELECT  b.BranchName,
        SUM(i.QuantityOnHand * p.UnitCost)           AS StockValue,
        SUM(sold.Units * p.UnitCost)                 AS COGS_12m,
        ROUND(SUM(sold.Units * p.UnitCost)
              / NULLIF(SUM(i.QuantityOnHand * p.UnitCost), 0), 2) AS StockTurn,
        ROUND(365 * SUM(i.QuantityOnHand * p.UnitCost)
              / NULLIF(SUM(sold.Units * p.UnitCost), 0), 0)      AS DaysOfCover
FROM      Inventory i
JOIN      Branch  b ON b.BranchID = i.BranchID
JOIN      Product p ON p.ProductID = i.ProductID
LEFT JOIN (
        SELECT od.ProductID, o.BranchID, SUM(od.Quantity) AS Units
        FROM    OrderDetails od
        JOIN    "Order" o ON o.OrderID = od.OrderID
        WHERE   o.OrderDate >= CURRENT_DATE - INTERVAL '12 months'
        GROUP BY od.ProductID, o.BranchID
) sold ON sold.ProductID = i.ProductID AND sold.BranchID = i.BranchID
GROUP BY b.BranchName
ORDER BY DaysOfCover DESC;

```

4.5 Supplier lead time and reliability

The Supplier entity earns its place here. Average lead time is the headline, but variance is what actually drives safety stock, so both are returned.

```

SELECT  s.SupplierName,
        COUNT(*)                               AS Orders,
        ROUND(AVG(o.ReceivedDate - o.OrderDate), 1) AS AvgLeadDays,
        ROUND(STDDEV_SAMP(o.ReceivedDate - o.OrderDate),1) AS LeadDayVariability,
        ROUND(100.0 * SUM(CASE WHEN o.ReceivedDate <= o.PromisedDate
                                THEN 1 ELSE 0 END) / COUNT(*), 1) AS OnTimePct
FROM      "Order" o
JOIN      Supplier s ON s.SupplierID = o.SupplierID
WHERE     o.ReceivedDate IS NOT NULL
GROUP BY s.SupplierName
HAVING   COUNT(*) >= 5
ORDER BY OnTimePct ASC, LeadDayVariability DESC;

```

4.6 Inter-branch imbalance

The query that turns five branches from five shops into one estate: find products that are below reorder point at one site while another site is holding surplus, so the answer is a transfer rather than a purchase order.

```

SELECT  p.SKU, p.ProductName,
        short.BranchName AS ShortAt, short.QuantityOnHand AS ShortQty,
        surplus.BranchName AS SurplusAt, surplus.QuantityOnHand AS SurplusQty
FROM      Product p
JOIN      (SELECT i.ProductID, b.BranchName, i.QuantityOnHand
        FROM Inventory i JOIN Branch b ON b.BranchID = i.BranchID
        WHERE i.QuantityOnHand < i.ReorderPoint) short
        ON short.ProductID = p.ProductID
JOIN      (SELECT i.ProductID, b.BranchName, i.QuantityOnHand
        FROM Inventory i JOIN Branch b ON b.BranchID = i.BranchID
        WHERE i.QuantityOnHand > i.ReorderPoint * 2) surplus
        ON surplus.ProductID = p.ProductID
ORDER BY p.SKU;

```

4.7 Count-versus-system variance

This is the query that connects the schema back to the evidence in the companion note: it is how the 59.54% discrepancy rate reported by ECR would be measured in this estate.² It requires the stock-count table proposed in Section 5.2.

```
SELECT  b.BranchName, p.SKU,
        i.QuantityOnHand          AS SystemQty,
        c.CountedQty,
        c.CountedQty - i.QuantityOnHand AS Variance,
        ROUND(100.0 * ABS(c.CountedQty - i.QuantityOnHand)
              / NULLIF(i.QuantityOnHand, 0), 1) AS VariancePct
FROM    StockCount c
JOIN    Inventory i ON i.ProductID = c.ProductID AND i.BranchID = c.BranchID
JOIN    Branch b   ON b.BranchID = c.BranchID
JOIN    Product p   ON p.ProductID = c.ProductID
WHERE   c.CountDate = (SELECT MAX(CountDate) FROM StockCount)
        AND c.CountedQty <> i.QuantityOnHand
ORDER BY ABS(c.CountedQty - i.QuantityOnHand) DESC;
```

4.8 Branch-level demand

Where the Customer entity pays off: branches serve different demand profiles, and estate-wide averages hide that.

```
SELECT  b.BranchName,
        COUNT(DISTINCT o.CustomerID)          AS ActiveCustomers,
        COUNT(DISTINCT o.OrderID)             AS Orders,
        ROUND(SUM(od.Quantity * od.UnitPrice), 2) AS Revenue,
        ROUND(SUM(od.Quantity * od.UnitPrice)
              / NULLIF(COUNT(DISTINCT o.OrderID), 0), 2) AS AvgOrderValue
FROM    "Order" o
JOIN    OrderDetails od ON od.OrderID = o.OrderID
JOIN    Branch b       ON b.BranchID = o.BranchID
WHERE   o.OrderDate >= CURRENT_DATE - INTERVAL '12 months'
GROUP BY b.BranchName
ORDER BY Revenue DESC;
```

5. WHAT TO BUILD NEXT

5.1 The movement event table

The schema's principal limitation, identified in Section 2, is that it records position without movement. The fix is one table.

```
CREATE TABLE InventoryMovement (
    MovementID BIGSERIAL PRIMARY KEY,
    ProductID  INT NOT NULL REFERENCES Product(ProductID),
    BranchID   INT NOT NULL REFERENCES Branch(BranchID),
    EmployeeID INT          REFERENCES Employee(EmployeeID),
    MovementType VARCHAR(20) NOT NULL
                CHECK (MovementType IN ('RECEIPT', 'SALE', 'TRANSFER_IN',
                                         'TRANSFER_OUT', 'ADJUSTMENT', 'WASTE', 'RETURN')),
    Quantity   INT NOT NULL,
    MovementTS TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    SourceRef  VARCHAR(50)
);
```

With that table, `QuantityOnHand` stops being a value that someone edits and becomes a derived figure that can be recomputed from its own history. That is the structural difference between a system that holds a number and a system that can explain one. It also makes the Employee foreign key meaningful: variance becomes attributable to a shift, a process step or a site rather than simply observed.

5.2 Stock counts and a reporting layer

A `StockCount` table (`ProductID`, `BranchID`, `CountDate`, `CountedQty`, `CountedBy`) enables the variance analysis in Section 4.7 and, over sequential counts, the accuracy-decay measurement that the ECR programme ran over 16 weeks.²

Beyond that, the operational schema above is correctly normalised for transaction processing and deliberately unsuited to reporting at volume. The standard next step is a separate dimensional layer: a periodic snapshot fact table holding stock position by product, branch and day, with conformed Product, Branch, Supplier and Date dimensions, and type-2 slowly changing dimensions so that a product reclassified mid-year does not silently rewrite last year's numbers. Keeping the two layers separate is the point; a single schema optimised for both does neither well.

6. MARKET CONTEXT FOR THE SKILL

An analysis of 855 data analyst job postings published in April 2026 found SQL named in **52.9%** of them, ahead of Excel at 50.5%, Python at 31.2%, Power BI at 29% and Tableau at 26.2%.³ A separate ongoing job-postings tracker puts SQL at 43% and reaches the same ordering, with SQL first.⁴ Both datasets are United States postings sourced primarily from Indeed, so the absolute percentages should not be read as United Kingdom figures; the consistent finding across both is the ranking, not the level.

The practical implication for an entry-level analyst is that SQL is the one technical skill where a hiring manager is most likely to ask for concrete evidence, and where a designed and implemented schema is stronger evidence than a completed course. The distinction that matters in interview is between having written queries against someone else's database and having decided what the tables should be, which is a design judgement rather than a syntax skill.

7. LIMITATIONS

- The AC51049 schema was implemented as an academic exercise for a five-branch scenario. It was not deployed, load-tested, or run against production data volumes, and no indexing or query-plan work is claimed.
- The ARTS comparison in Section 2 is a subject-area mapping made from the model's published description.¹ It is not a formal conformance assessment against the ARTS specification.
- The queries in Section 4 are written in ANSI-leaning SQL with PostgreSQL syntax for intervals, window functions and date arithmetic. Sections 4.4 and 4.5 use date subtraction and `INTERVAL` forms that require adjustment on SQL Server or MySQL.
- Sections 4.7 and 5.2 depend on a `StockCount` table that is proposed here rather than implemented in the original project.
- The job-market figures in Section 6 are United States postings and are cited for relative ranking only.

8. SOURCES

1. Object Management Group, Retail Domain Technology Committee. *ARTS Operational Data Model*, version 7.3. Originally developed by the Association for Retail Technology Standards under the National Retail Federation.
<https://www.omg.org/retail/operational-data-model.htm>

2. Rekik, Y., Syntetos, A. and Glock, C. (2020). *Measuring the Sales Impact of Improving Inventory Records*. ECR Retail Loss. 7 European retailers, 100 stores, ~1m SKUs examined.
<https://www.ecrloss.com/research/grow-sales-by-improving-inventory-records/>
3. 365 Data Science (April 2026). *Data Analyst Job Market in 2026: Demands and Prospects*. 1,000 Indeed US postings collected, 855 retained after preprocessing.
<https://365datascience.com/career-advice/the-data-analyst-job-market/>
4. datanerd.tech. *Top Data Analyst Skills in 2026*. Ongoing job-postings tracker.
<https://datanerd.tech/research/data-analyst-skills>

Prepared as a companion note to the Relational Database Design (AC51049) project, University of Dundee. The schema discussed is the eight-entity model implemented for that project; the movement, stock-count and dimensional extensions in Section 5 are proposed, not built.